

Full-Stack Integration

NODE.JS + MySQL

Building the Books REST API

Complete Step-by-Step Live Coding Session

WHAT THIS TUTORIAL COVERS

1. **Project Setup** - npm init, install packages
2. **MySQL Connection** - mysql2 driver, connection pool
3. **GET All Books** - first endpoint - fetch data
4. **GET Book by ID** - URL parameters, 404 handling
5. **POST New Book** - create with req.body
6. **PUT Update Book** - full replacement
7. **PATCH Partial Update** - dynamic query building
8. **Testing and Reference** - Thunder Client complete guide

Complete Live Session Documentation

Real Commands, Real Errors, Real Solutions

Introduction

What This Tutorial Is

This tutorial documents a complete Node.js and MySQL integration session. It covers every step from creating a new project folder to building 5 REST API endpoints that connect to a real MySQL database. Every command, code snippet, and error solution shown here comes from actual live coding.

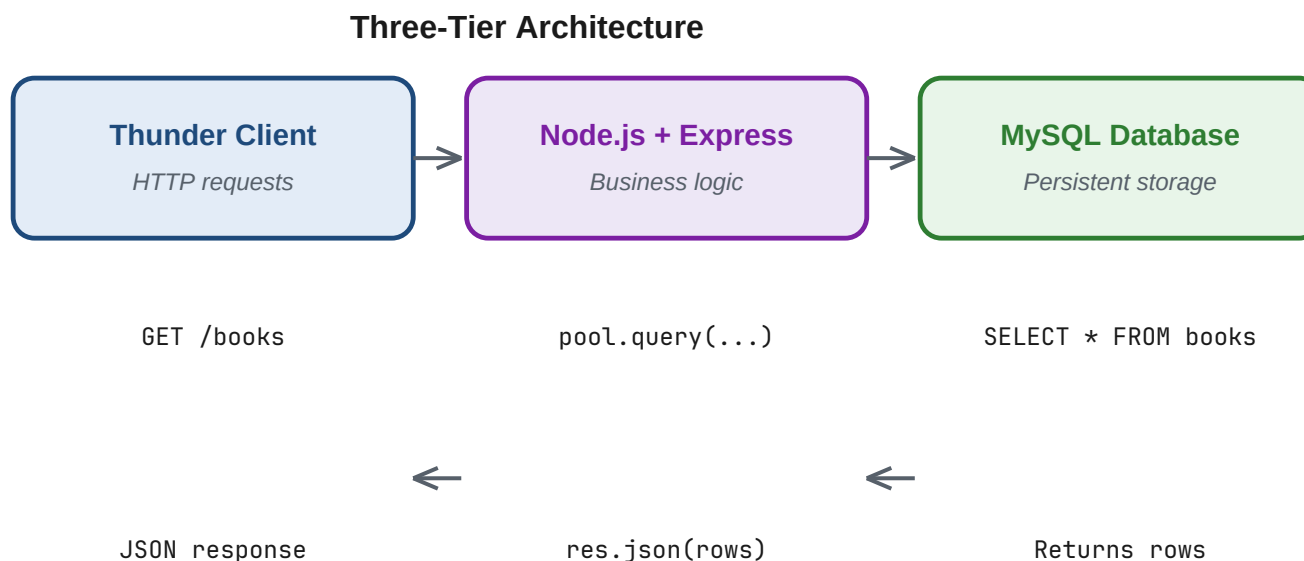
What You Will Build

A complete REST API for managing books with these endpoints:

- **GET /books** - Fetch all books from database
- **GET /books/:id** - Fetch one specific book
- **POST /books** - Add a new book
- **PUT /books/:id** - Update entire book
- **PATCH /books/:id** - Update selected fields only

Every endpoint reads from or writes to a real MySQL database. Data persists across server restarts and can be accessed from multiple clients simultaneously.

The Three-Tier Architecture



This is what we built today

A complete data flow from client through server to database

All 5 CRUD endpoints follow this same pattern

Figure 1: The complete data flow across three tiers

Prerequisites

Before starting this tutorial, you should have:

- **Node.js installed** - version 18 or higher
- **MySQL installed and running** - Community Edition 8.0+
- **MySQL Workbench** or Command Line access
- **VS Code** or any code editor
- **Thunder Client extension** installed in VS Code
- **Basic knowledge of Express.js** - routes, req, res
- **Basic SQL knowledge** - SELECT, INSERT, UPDATE, DELETE
- **A database called my_first_db** with a books table

The Books Table Schema

The database table this tutorial expects looks like this:

books table structure

```
CREATE TABLE books (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  author VARCHAR(100) NOT NULL,  
  genre VARCHAR(50),  
  pages INT,  
  rating INT  
);
```

With at least 15 books pre-loaded:

Sample data insert

```
INSERT INTO books (title, author, genre, pages, rating) VALUES  
  ('Wings of Fire', 'APJ Abdul Kalam', 'biography', 180, 9),  
  ('Sapiens', 'Yuval Noah Harari', 'history', 464, 10),  
  ('Atomic Habits', 'James Clear', 'self-help', 320, 8),  
  ('The Alchemist', 'Paulo Coelho', 'fiction', 208, 7),  
  ('1984', 'George Orwell', 'fiction', 328, 9);  
-- ... 10 more books ...
```

Book Data: This tutorial uses real books - Wings of Fire, Sapiens, Atomic Habits, and others. If your table has different books, the code still works - just the results will differ.

How to Use This Tutorial

- **Follow chapters in order** - each builds on the previous
- **Type every command** - do not copy-paste
- **Test after each endpoint** - do not add multiple at once
- **Read the code explanations** - understand every line
- **Fix errors as they appear** - Chapter 8 has troubleshooting

Time Estimate

Chapter	Estimated Time
Chapter 1: Project Setup	20 min
Chapter 2: MySQL Connection	20 min
Chapter 3: GET All Books	20 min
Chapter 4: GET Book by ID	20 min
Chapter 5: POST New Book	30 min
Chapter 6: PUT Update Book	20 min
Chapter 7: PATCH Partial Update	30 min
Chapter 8: Testing and Reference	20 min
TOTAL	3 hours

1

CHAPTER 1: Project Setup

Creating the folder and installing packages

Every Node.js project starts with a folder and some installed packages. This chapter creates the workspace where all coding will happen.

The Goal

By the end of this chapter, you will have:

- A new project folder called **books-api**
- A **package.json** file identifying your project
- Two installed packages: **express** and **mysql2**
- A **node_modules** folder with dependencies

Step 1

Open VS Code and Terminal

First, prepare your environment:

1. Open VS Code from Start menu
2. Open the integrated terminal (Ctrl + backtick, or Terminal menu)
3. The terminal will show your current directory

PowerShell vs Command Prompt: VS Code opens PowerShell by default on Windows. This is fine. If you see permission errors later, we will fix them in a troubleshooting section.

Step 2

Navigate to Your Workspace

Choose a location for your projects. In this tutorial, we use D:\MCA-3rd Unit\SQLwithNodejs. Adjust the path for your system.

Change to your workspace folder

```
PS C:\Users\Anurag> D:
PS D:\> cd MCA-3rd Unit\SQLwithNodejs
PS D:\MCA-3rd Unit\SQLwithNodejs>
```

Step 3

Create the Project Folder

Create books-api folder

```
PS D:\MCA-3rd Unit\SQLwithNodejs> mkdir books-api
```

No output means success. The folder is created.

Step 4

Enter the New Folder

Change into books-api

```
PS D:\MCA-3rd Unit\SQLwithNodejs> cd books-api  
PS D:\MCA-3rd Unit\SQLwithNodejs\books-api>
```

Prompt Change: Notice how the prompt now shows books-api at the end. This confirms you are inside the project folder. All future commands will run here.

Step 5**Initialize Node.js Project**

The command `npm init` creates a `package.json` file - the identity document of your project. The `-y` flag uses default answers for all questions.

Initialize project

```
PS D:\MCA-3rd Unit\SQLwithNodejs\books-api> npm init -y
```

Expected Output

Wrote to D:\MCA-3rd Unit\SQLwithNodejs\books-api\package.json:

```
{
  "name": "books-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Troubleshooting - Execution Policy Error

On some Windows systems, you may see this error:

PowerShell Error

```
npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because
running scripts is disabled on this system. For more information, see
about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170.
```

Fix this with a one-time command:

Fix execution policy

```
PS ...> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

When prompted **[Y] Yes [A] Yes to All [N] No [L] No to All**, type **Y** and press Enter. Then retry `npm init -y`.

What This Fix Does: This tells Windows to allow trusted scripts (like npm) to run. Downloaded malicious scripts still blocked - only signed and trusted ones allowed. Safe balance for developers.

Step 6**Install Express**

Express is the web framework you already know. Install it as a dependency:

Install express

```
PS ...\\books-api> npm install express
```

Expected Output (numbers may vary)

```
added 66 packages, and audited 67 packages in 8s
```

```
12 packages are looking for funding  
run `npm fund` for details
```

```
found 0 vulnerabilities
```

Step 7

Install mysql2 Driver

This is the critical package that enables Node.js to communicate with MySQL. Without it, Node.js cannot connect to MySQL.

Install mysql2

```
PS ...\\books-api> npm install mysql2
```

Expected Output

```
added 12 packages, and audited 79 packages in 5s
```

```
found 0 vulnerabilities
```

Why mysql2 and not mysql?: The mysql package is older and uses callbacks. mysql2 is faster, actively maintained, and supports modern async/await syntax. Always prefer mysql2 for new projects.

Step 8**Verify Installation**

Check that everything is properly installed:

List files

```
PS ...\books-api> dir
```

Expected Output

```
Directory: D:\MCA-3rd Unit\SQLwithNodejs\books-api
```

```
Mode LastWriteTime Length Name
----
d----- 17-08-2025 10:00 node_modules
-a--- 17-08-2025 10:00 1234 package-lock.json
-a--- 17-08-2025 10:00 305 package.json
```

The three items confirm successful setup:

- **node_modules/** - folder with installed packages
- **package.json** - your project identity
- **package-lock.json** - exact versions locked

Final Project Structure**Final Project Folder Structure**

```
books-api/
|
|-- node_modules/
|       (contains express, mysql2, dependencies)
|
|-- books.js
|       (main API file - your code)
|
|-- package.json
|       (project info & dependency list)
|
`-- package-lock.json
```

Figure 2: Complete folder structure after setup

Chapter 1 Summary

You have successfully:

- Created a new project folder called books-api
- Initialized it with `npm init -y`

- Created a package.json identifying your project
- Installed Express web framework
- Installed mysql2 driver for MySQL communication
- Verified the installation with dir command

The project is now ready for coding. Next chapter starts writing the actual code that connects to MySQL.

2

CHAPTER 2: MySQL Connection

Setting up the connection pool with mysql2

This chapter creates the code that establishes communication between Node.js and MySQL. This is the most conceptually important chapter - understanding it makes everything else straightforward.

The Goal

- Create the main file **books.js**
- Import Express and mysql2
- Create a MySQL connection pool
- Configure it to connect to **my_first_db** database
- Add JSON middleware for parsing request bodies

Step 1

Create the Main File

In VS Code, create a new file in the books-api folder:

1. Right-click in the file explorer panel (left sidebar)
2. Select **New File**
3. Type **books.js** (exact spelling, lowercase, with .js extension)
4. Press Enter

Critical: File Location: Make sure books.js is INSIDE the books-api folder, not outside. The file must be next to package.json. If created elsewhere, drag it into books-api or delete and recreate.

Step 2

Import Express

Type these two lines (do not copy-paste):

books.js - Express import

```
const express = require('express');  
const app = express();
```

These lines are familiar from previous Express projects:

- **Line 1:** Import the Express framework
- **Line 2:** Create a new Express app instance

Step 3

Import mysql2

Below the Express imports, add:

books.js - MySQL import

```
const mysql = require('mysql2/promise');
```

This line has an important detail:

Part	What It Means
<code>require</code>	Standard Node.js import function
<code>'mysql2/promise'</code>	Modern async/await version (NOT just "mysql2")
<code>const mysql</code>	Store imported module in mysql variable
Why /promise Matters: Without /promise, mysql2 uses old callback style which is complex and error-prone. With /promise, you get modern async/await syntax that reads like regular code.	

Step 4 Create Connection Pool

The most critical piece - the connection pool:

books.js - Connection pool

```
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'YOUR_PASSWORD_HERE',
  database: 'my_first_db'
});
```

Replace YOUR_PASSWORD_HERE: Change YOUR_PASSWORD_HERE to your actual MySQL root password - the one you set during MySQL installation. Without the correct password, connection will fail.

Understanding Each Configuration Option

Option	Meaning	Common Value
host	Where MySQL server is running	localhost
user	MySQL username to connect as	root
password	Password for that user	Set during install
database	Which database to use	my_first_db

What Is a Connection Pool?

A pool is a group of ready-to-use MySQL connections. Instead of creating a new connection for every request (slow!), the pool maintains multiple connections that can be reused. When your route handler needs to run a query, it borrows a connection from the pool, uses it, then returns it. This dramatically improves performance.

Step 5**Add JSON Middleware**

Below the pool, add:

books.js - Middleware

```
app.use(express.json());
```

This middleware parses JSON request bodies. Without it, req.body will be undefined in POST/PUT/PATCH routes. This is the same middleware used in Students API.

Step 6**Save the File**

Press **Ctrl + S** to save. The white dot on the file tab should disappear, confirming save.

Current State of books.js

At this point, your books.js should look exactly like this:

Complete books.js (Chapter 2 end)

```
const express = require('express');
const app = express();

const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'YOUR_ACTUAL_PASSWORD',
  database: 'my_first_db'
});

app.use(express.json());
```

Chapter 2 Summary

You have successfully:

- Created the main file books.js
- Imported Express framework
- Imported mysql2 with promise support
- Created a connection pool with your MySQL credentials
- Added JSON parsing middleware

Not Runnable Yet: The file is not runnable yet because there is no route handler or app.listen(). We add those in the next chapters. Do not try node books.js here - it will exit silently.

3

CHAPTER 3: GET All Books

The first endpoint - fetching data from MySQL

This chapter creates the first REST endpoint that actually reads data from MySQL. Once this works, the historic moment happens - Node.js talks to MySQL for the first time!

The Endpoint

GET

`/books`

This endpoint returns all books stored in the database as a JSON array. It corresponds to the SQL query `SELECT * FROM books`.

Step 1**Add the Route Handler**

Below the middleware line, add this route:

books.js - GET all books

```
app.get('/books', async (req, res) => {
  const [rows] = await pool.query('SELECT * FROM books');
  res.json(rows);
});
```

Understanding the Code

Line 1: The Route Declaration

The route signature

```
app.get('/books', async (req, res) => {
```

- **app.get** - handles GET requests (familiar)
- **'/books'** - the URL path (familiar)
- **async** - NEW! Marks this function as asynchronous
- **(req, res)** - request and response objects (familiar)

Why async?: MySQL queries take time to execute (milliseconds, but still non-zero). Node.js needs to wait for the response. The `async` keyword enables the use of `await` inside the function - required for waiting on database queries.

Line 2: The Database Query

The query line

```
const [rows] = await pool.query('SELECT * FROM books');
```

This line has three critical parts:

Part	Meaning
pool.query(...)	Send SQL to MySQL and get response
await	NEW! Wait for query to complete before continuing
const [rows]	NEW! Destructuring - extract first item from returned array

Why Destructuring [rows]?

The pool.query() function returns an array with two items: [rows, fields]. Only the first item (rows) is needed - fields contains metadata rarely used. The destructuring const [rows] extracts just the first item.

Alternative (not destructured)

```
// This works but is uglier
const result = await pool.query('SELECT * FROM books');
const rows = result[0];
```

Line 3: The Response

Send JSON response

```
res.json(rows);
```

This line is familiar - send the rows array as JSON to the client. The exact same pattern as Students API, except now data comes from MySQL instead of a JavaScript array.

Step 2**Add Server Startup Code**

At the bottom of the file, add the `app.listen()` code:

books.js - Server startup

```
app.listen(3001, () => {  
  console.log('Server running on port 3001');  
  console.log('Books API ready - connected to MySQL!');  
});
```

Familiar pattern. The extra log message helps confirm both server and MySQL connection are ready.

Step 3**Save the File**

Press **Ctrl + S**. Ensure white dot disappears.

Step 4**Start the Server**

In terminal, run:

Start the API

```
PS D:\...\books-api> node books.js
```

Expected Output

```
Server running on port 3001  
Books API ready - connected to MySQL!
```

Cursor Stops Here: After the messages, the terminal cursor waits with no prompt. This is CORRECT! The server is running and listening for requests. Do not close the terminal.

Step 5**Test with Thunder Client**

In VS Code, open Thunder Client (lightning bolt icon):

1. Click **New Request**
2. Method: **GET** (default)
3. URL: `http://localhost:3001/books`
4. Click **Send**

Expected Response

Status: **200 OK** (green)

Body: An array of book objects, for example:

Thunder Client response (partial)

```
[
  {
    "id": 1,
    "title": "Wings of Fire",
    "author": "APJ Abdul Kalam",
    "genre": "biography",
    "pages": 180,
    "rating": 9
  },
  {
    "id": 2,
    "title": "Sapiens",
    "author": "Yuval Noah Harari",
    "genre": "history",
    "pages": 464,
    "rating": 10
  }
  // ... 13 more books
]
```

What Just Happened

This is the moment where full-stack development becomes real:

1. Thunder Client sent HTTP GET request to your server
2. Express received the request and matched /books route
3. Route handler executed the SQL query
4. MySQL fetched all 15 rows from books table
5. Rows returned to Node.js as JavaScript objects
6. Node.js sent them back as JSON to Thunder Client
7. Thunder Client displayed the data

All of this happens in milliseconds. And the data is REAL - stored in MySQL on disk. Restart the server, restart the computer, the data stays. This is production-quality data persistence.

Chapter 3 Summary

- Created first route handler with async/await
- Successfully queried MySQL from Node.js
- Tested with Thunder Client
- Data returned from real database
- Server persists across restarts

4

CHAPTER 4: GET Book by ID

URL parameters and 404 error handling

This chapter creates an endpoint that fetches ONE specific book by its ID. Two new concepts appear: URL parameters and prepared statements for security.

The Endpoint

GET**/books/:id**

Notice :id - this is a URL parameter that varies. The client sends /books/1, /books/2, etc.

Step 1**Stop the Server**

In terminal where server is running, press **Ctrl + C**.

If prompted "**Terminate batch job (Y/N)?**", type **Y** and Enter.

Why Stop?: Node.js does not auto-reload when you change code. After every code change, you must stop and restart the server for changes to take effect. Tools like nodemon automate this, but for learning, manual restart teaches the workflow.

Step 2**Add the Route Handler**

Below the GET /books route, add this new endpoint:

books.js - GET book by ID

```
app.get('/books/:id', async (req, res) => {
  const id = parseInt(req.params.id);
  const [rows] = await pool.query('SELECT * FROM books WHERE id = ?', [id]);

  if (rows.length === 0) {
    return res.status(404).json({ message: 'Book not found' });
  }

  res.json(rows[0]);
});
```

Understanding the Code

URL Parameter :id**Extract the ID from URL**

```
const id = parseInt(req.params.id);
```

- **req.params.id** - gets the value from URL (e.g. "1" from /books/1)
- **parseInt(...)** - converts string to number
- URL params are ALWAYS strings, but MySQL id is a number

The Prepared Statement

Query with placeholder

```
const [rows] = await pool.query(  
  'SELECT * FROM books WHERE id = ?',  
  [id]  
);
```

This uses TWO important new features:

- **?** - a placeholder in the SQL string
- **[id]** - actual value that fills the placeholder

SQL Injection Prevention: NEVER concatenate user input directly into SQL strings. Using placeholders (?) with the values array lets MySQL safely handle the substitution. This prevents SQL injection attacks - one of the most common security vulnerabilities.

DANGEROUS Alternative (Never Do This)

SECURITY VULNERABILITY - do not use

```
// If id is "1 OR 1=1", this returns ALL books  
const [rows] = await pool.query(  
  `SELECT * FROM books WHERE id = ${id}`  
);
```

The 404 Check

Handle book not found

```
if (rows.length === 0) {  
  return res.status(404).json({ message: 'Book not found' });  
}
```

- **rows.length === 0** - empty array means nothing matched
- **res.status(404)** - HTTP status "Not Found"
- **return** - CRITICAL! Stops execution here
- Without return, the next `res.json()` would also try to execute

The Success Response

Return single book

```
res.json(rows[0]);
```

MySQL always returns an array. But we asked for one specific book, so return just `rows[0]` - the first (and only) element.

Step 3**Save and Restart**

Save (**Ctrl + S**) and restart:

Restart server

```
PS D:\...\books-api> node books.js
Server running on port 3001
Books API ready - connected to MySQL!
```

Step 4**Test in Thunder Client****Test 1: Valid ID**

- Method: **GET**
- URL: `http://localhost:3001/books/1`
- Click **Send**

Expected response - just one book:

Response for id=1

```
{
  "id": 1,
  "title": "Wings of Fire",
  "author": "APJ Abdul Kalam",
  "genre": "biography",
  "pages": 180,
  "rating": 9
}
```

Status: **200 OK**

Test 2: Different ID

Try `http://localhost:3001/books/5`

Should return the book at id=5.

Test 3: Invalid ID (404 Test)

Try `http://localhost:3001/books/999`

Expected 404 response

```
{
  "message": "Book not found"
}
```

Status: **404 Not Found**

Why This Test Matters: Real APIs must handle invalid inputs gracefully. Without the 404 check, the API would return an empty array which confuses clients. Proper error handling is what separates toy projects from production APIs.

Chapter 4 Summary

- Used URL parameters with :id syntax
- Extracted params with req.params.id
- Converted string to number with parseInt
- Used prepared statements (?) for security
- Handled not-found case with 404 status
- Returned single object with rows[0]

5

CHAPTER 5: POST New Book

Creating new records with req.body

This chapter creates the endpoint that adds new books to MySQL. Any book added stays permanently in the database. This is where the API becomes truly interactive.

The Endpoint

POST**/books**

Notice - no :id in URL. New books get IDs auto-generated by MySQL via AUTO_INCREMENT.

Step 1**Stop the Server**

Ctrl + C in terminal, then **Y** to confirm.

Step 2**Add the Route Handler**

Below the GET /books/:id route, add:

books.js - POST new book

```
app.post('/books', async (req, res) => {
  const { title, author, genre, pages, rating } = req.body;

  const [result] = await pool.query(
    'INSERT INTO books (title, author, genre, pages, rating) VALUES (?, ?, ?, ?, ?)',
    [title, author, genre, pages, rating]
  );

  res.status(201).json({
    id: result.insertId,
    title,
    author,
    genre,
    pages,
    rating
  });
});
```

Understanding the Code

Extract Data from Request Body

Destructure request body

```
const { title, author, genre, pages, rating } = req.body;
```

- **req.body** - the JSON payload sent by client
- **{ title, author, ... }** - destructuring extracts 5 fields

- This works because `app.use(express.json())` parsed it

Without JSON Middleware: If you forgot `app.use(express.json())`, then `req.body` would be undefined and this destructuring would crash. Always include the middleware.

The INSERT Query

Insert with placeholders

```
const [result] = await pool.query(
  'INSERT INTO books (title, author, genre, pages, rating) VALUES (?, ?, ?, ?, ?)',
  [title, author, genre, pages, rating]
);
```

Key points about this query:

- **Five placeholders (?)** - one for each field
- **Five values in array** - MUST match the ? order
- Order in query: title, author, genre, pages, rating
- Order in array: title, author, genre, pages, rating
- MySQL auto-generates the id (AUTO_INCREMENT)

The result Object

INSERT queries return a special result object with useful information:

Property	Meaning
<code>result.insertId</code>	The auto-generated ID of the new row
<code>result.affectedRows</code>	Number of rows added (should be 1)
<code>result.warningStatus</code>	Any warnings from MySQL

The `result.insertId` is especially valuable - it tells you the ID MySQL assigned to your new row. Return this to the client so they know the ID for future operations.

The Response

Return created resource

```
res.status(201).json({
  id: result.insertId,
  title,
  author,
  genre,
  pages,
  rating
});
```

- **res.status(201)** - HTTP 201 means "Created" (POST standard)
- **id: result.insertId** - the new auto-generated ID
- Other fields returned as-is (from destructured values)
- Shorthand: title means title: title

Step 3

Save and Restart

Save (**Ctrl + S**) and restart:

Restart server

```
PS D:\...\books-api> node books.js
Server running on port 3001
Books API ready - connected to MySQL!
```

Step 4

Test in Thunder Client

Setup the Request

1. New Request
2. Change Method to **POST**
3. URL: `http://localhost:3001/books` (no ID!)
4. Click **Body** tab
5. Select **Json** option

Enter the JSON Body

Request body (paste in Body tab)

```
{
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald",
  "genre": "fiction",
  "pages": 180,
  "rating": 9
}
```

Notice:

- All keys have **double quotes** (JSON is strict)

- Text values in quotes (title, author, genre)
- Numbers without quotes (pages, rating)
- **NO id field** - MySQL will generate it

Send and Check Response

Click **Send**. Expected response:

Response (status 201)

```
{
  "id": 16,
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald",
  "genre": "fiction",
  "pages": 180,
  "rating": 9
}
```

- **id: 16** - MySQL auto-generated this (assuming 15 books existed)
- Other fields match what you sent
- Status: **201 Created**

Step 5

Verify Persistence

Three ways to verify the book was actually saved:

Method 1: GET All Books

Send GET /books in Thunder Client. Should now show 16 books instead of 15.

Method 2: GET by New ID

Send GET /books/16. Should return The Great Gatsby.

Method 3: MySQL Command Line

Verify in MySQL directly

```
mysql> USE my_first_db;
mysql> SELECT * FROM books WHERE id = 16;
```

MySQL response

```
+-----+-----+-----+-----+-----+-----+
| id | title | author | genre | pages | rating |
+-----+-----+-----+-----+-----+-----+
| 16 | The Great Gatsby | F. Scott Fitzgerald | fiction | 180 | 9 |
+-----+-----+-----+-----+-----+-----+
```

True Persistence Confirmed: The book appears in Thunder Client AND MySQL Command Line. This proves the data is not in memory - it is on disk. Restart your server, restart your computer, the book remains. This is real production-quality persistence.

Chapter 5 Summary

- Created POST endpoint for adding new books
- Extracted data from req.body via destructuring
- Used INSERT query with 5 placeholders
- Returned auto-generated ID via result.insertId
- Used HTTP 201 Created status
- Verified persistence in MySQL directly

6

CHAPTER 6: PUT Update Book

Complete replacement of an existing book

This chapter creates the endpoint for updating books. PUT replaces the ENTIRE book - all fields must be provided.

The Endpoint

PUT**/books/:id**

PUT combines URL parameter (id) with request body (new data). The id determines which book to update.

Step 1**Stop the Server**

Ctrl + C and **Y** to confirm.

Step 2**Add the Route Handler**

Below the POST route, add:

books.js - PUT update book

```
app.put('/books/:id', async (req, res) => {
  const id = parseInt(req.params.id);
  const { title, author, genre, pages, rating } = req.body;

  const [result] = await pool.query(
    'UPDATE books SET title = ?, author = ?, genre = ?, pages = ?, rating = ? WHERE id = ?',
    [title, author, genre, pages, rating, id]
  );

  if (result.affectedRows === 0) {
    return res.status(404).json({ message: 'Book not found' });
  }

  res.json({
    id,
    title,
    author,
    genre,
    pages,
    rating
  });
});
```

Understanding the Code

Getting Both URL Parameter and Body

Get id from URL, data from body

```
const id = parseInt(req.params.id);  
const { title, author, genre, pages, rating } = req.body;
```

PUT is unique - it uses both:

- **URL parameter (id)** - which book to update
- **Body** - what the new data should be

The UPDATE Query**The critical UPDATE query**

```
const [result] = await pool.query(  
  'UPDATE books SET title = ?, author = ?, genre = ?, pages = ?, rating = ? WHERE id = ?',  
  [title, author, genre, pages, rating, id]  
);
```

This query has SIX placeholders:

- Five for SET clause (new field values)
- One for WHERE clause (which row to update)
- Values array must include id at the end

WHERE Clause is CRITICAL: Without the WHERE clause, this UPDATE would modify ALL books to the same values! This is one of the most catastrophic SQL mistakes possible. Always include WHERE in UPDATE queries.

Checking If Book Existed

404 handling for update

```
if (result.affectedRows === 0) {  
    return res.status(404).json({ message: 'Book not found' });  
}
```

- **result.affectedRows** - how many rows were modified
- If 0, no book had that id
- Return 404 with clear message

Success Response

Return updated book

```
res.json({  
    id,  
    title,  
    author,  
    genre,  
    pages,  
    rating  
});
```

Return the complete updated resource. Client can verify the change without another GET request.

Step 3

Save, Restart, Test

Save file, restart server:

Restart

```
PS D:\...\books-api> node books.js
```

Test 1: Update Gatsby

- Method: **PUT**
- URL: `http://localhost:3001/books/16`
- Body tab: Json

PUT request body

```
{  
    "title": "The Great Gatsby",  
    "author": "F. Scott Fitzgerald",  
    "genre": "classic fiction",  
    "pages": 218,  
    "rating": 10  
}
```

Notice: genre changed to "classic fiction", pages to 218, rating to 10.

Click **Send**. Response:

Expected response

```
{
  "id": 16,
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald",
  "genre": "classic fiction",
  "pages": 218,
  "rating": 10
}
```

Status: **200 OK**

Test 2: Verify with GET

Send GET /books/16 to confirm changes persisted.

Test 3: 404 Test

Try PUT /books/999 with any body. Should return 404 error.

Limitations of PUT

PUT Requires ALL Fields: PUT is "full replacement". If you send only some fields, the others will be set to NULL or undefined. For partial updates (like changing just the rating), use PATCH instead - covered in the next chapter.

Chapter 6 Summary

- Created PUT endpoint for full replacement
- Combined URL parameter with request body
- UPDATE query with 6 placeholders
- Checked affectedRows for 404 handling
- Returned updated resource in response

7

CHAPTER 7: PATCH Partial Update

Update only selected fields with dynamic queries

This chapter creates the most sophisticated endpoint. PATCH allows updating only the fields you want to change - a common requirement in real APIs. The implementation involves dynamic query building.

PATCH vs PUT

Aspect	PUT	PATCH
Purpose	Full replacement	Partial update
Body required	All fields	Only fields to change
Missing fields	Set to NULL	Left unchanged
Common use	Complete edit form	Toggle status, rename, etc.

The Endpoint

PATCH`/books/:id`**Step 1****Stop the Server****Step 2****Add the Route Handler**

Below the PUT route, add:

```
books.js - PATCH endpoint
```

```
app.patch('/books/:id', async (req, res) => {
  const id = parseInt(req.params.id);
  const updates = req.body;

  if (Object.keys(updates).length === 0) {
    return res.status(400).json({ message: 'No fields to update' });
  }

  const fields = Object.keys(updates).map(key => `${key} = ?`).join(', ');
  const values = Object.values(updates);
  values.push(id);

  const [result] = await pool.query(
    `UPDATE books SET ${fields} WHERE id = ?`,
    values
  );

  if (result.affectedRows === 0) {
    return res.status(404).json({ message: 'Book not found' });
  }

  const [rows] = await pool.query('SELECT * FROM books WHERE id = ?', [id]);
  res.json(rows[0]);
});
```

Understanding the Code

Get Body as Object

Get all updates as object

```
const updates = req.body;
```

Different from PUT - we do NOT destructure specific fields. We do not know which fields the client will send.

Empty Body Check

Reject empty updates

```
if (Object.keys(updates).length === 0) {
  return res.status(400).json({ message: 'No fields to update' });
}
```

- Empty body {} means no changes requested
- Return HTTP 400 (Bad Request) - client error
- Different from 404 - request itself is malformed

Dynamic Query Building - THE MAGIC

These three lines are the most important in this tutorial:

Build query dynamically

```
const fields = Object.keys(updates).map(key => `${key} = ?`).join(', ');
const values = Object.values(updates);
values.push(id);
```

Walk through with example. Suppose body is:

Example request body

```
{ "rating": 8, "pages": 250 }
```

Step 1: Get Field Names

Object.keys(updates)

```
Object.keys(updates)
// Result: ["rating", "pages"]
```

Step 2: Convert to SQL Fragments

map to key = ? format

```
.map(key => `${key} = ?`)
// Result: ["rating = ?", "pages = ?"]
```

Step 3: Join with Commas

join with comma

```
.join(", ")
// Result: "rating = ?, pages = ?"
```

Step 4: Get Values Array

Object.values(updates)

```
Object.values(updates)
// Result: [8, 250]
```

Step 5: Add ID at End

Push id for WHERE clause

```
values.push(id);
// Result: [8, 250, 16] (assuming id = 16)
```

Final Query

The template literal builds:

Dynamically constructed query

```
UPDATE books SET rating = ?, pages = ? WHERE id = ?  
-- With values: [8, 250, 16]
```

This is Production-Grade Code: This dynamic query building is exactly how real APIs handle partial updates. Understanding it means you can adapt this pattern to any table with any columns.

Return Updated Book**Fetch and return updated record**

```
const [rows] = await pool.query('SELECT * FROM books WHERE id = ?', [id]);  
res.json(rows[0]);
```

Unlike PUT (which returns known values), PATCH does not know the complete updated state. So we fetch the row after update to return the full current state.

Step 3**Save, Restart, Test**

Save file, then:

Restart

```
PS D:\...\books-api> node books.js
```

Test 1: Update Only Rating

- Method: **PATCH**
- URL: `http://localhost:3001/books/16`
- Body (Json):

PATCH request - single field

```
{
  "rating": 8
}
```

Click Send. Response should return complete book:

Response

```
{
  "id": 16,
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald",
  "genre": "classic fiction",
  "pages": 218,
  "rating": 8
}
```

Notice: only rating changed. Everything else stayed the same.

Test 2: Update Multiple Fields**PATCH multiple fields**

```
{
  "genre": "American classic",
  "pages": 220
}
```

Response shows both changes, but title/author/rating stay same.

Test 3: Empty Body (Should Fail)

Send PATCH with empty body `{}`. Expected response:

Response for empty body

```
{
  "message": "No fields to update"
}
```

Status: **400 Bad Request**

Test 4: Invalid ID (404)

Send PATCH to /books/999 with any body. Expected 404 response.

Chapter 7 Summary

- Created PATCH endpoint for partial updates
- Handled dynamic body without knowing exact fields
- Built SQL query dynamically with Object.keys/values
- Used template literals for query construction
- Fetched updated row for complete response
- Returned 400 for empty body, 404 for missing book

8

CHAPTER 8: Testing and Reference

Complete Thunder Client guide and full code listing

This chapter provides reference materials - the complete final code, Thunder Client testing guide, and troubleshooting for common issues.

Complete Final books.js

Here is the entire file with all 5 endpoints. Use this as a reference to compare with your code:

books.js - Part 1: Setup

```
const express = require('express');
const app = express();

const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'YOUR_PASSWORD',
  database: 'my_first_db'
});

app.use(express.json());
```

books.js - Part 2: GET routes

```
// GET all books
app.get('/books', async (req, res) => {
  const [rows] = await pool.query('SELECT * FROM books');
  res.json(rows);
});

// GET one book by ID
app.get('/books/:id', async (req, res) => {
  const id = parseInt(req.params.id);
  const [rows] = await pool.query(
    'SELECT * FROM books WHERE id = ?', [id]
  );
  if (rows.length === 0) {
    return res.status(404).json({ message: 'Book not found' });
  }
  res.json(rows[0]);
});
```

books.js - Part 3: POST route

```
// POST - create new book
app.post('/books', async (req, res) => {
  const { title, author, genre, pages, rating } = req.body;
  const [result] = await pool.query(
    'INSERT INTO books (title, author, genre, pages, rating) VALUES (?, ?, ?, ?, ?)',
    [title, author, genre, pages, rating]
  );
  res.status(201).json({
    id: result.insertId,
    title, author, genre, pages, rating
  });
});
```

books.js - Part 4: PUT route

```
// PUT - full update
app.put('/books/:id', async (req, res) => {
  const id = parseInt(req.params.id);
  const { title, author, genre, pages, rating } = req.body;
  const [result] = await pool.query(
    'UPDATE books SET title=?, author=?, genre=?, pages=?, rating=? WHERE id=?',
    [title, author, genre, pages, rating, id]
  );
  if (result.affectedRows === 0) {
    return res.status(404).json({ message: 'Book not found' });
  }
  res.json({ id, title, author, genre, pages, rating });
});
```

books.js - Part 5: PATCH route

```
// PATCH - partial update
app.patch('/books/:id', async (req, res) => {
  const id = parseInt(req.params.id);
  const updates = req.body;
  if (Object.keys(updates).length === 0) {
    return res.status(400).json({ message: 'No fields to update' });
  }
  const fields = Object.keys(updates).map(key => `${key} = ?`).join(', ');
  const values = Object.values(updates);
  values.push(id);
  const [result] = await pool.query(
    `UPDATE books SET ${fields} WHERE id = ?`, values
  );
  if (result.affectedRows === 0) {
    return res.status(404).json({ message: 'Book not found' });
  }
  const [rows] = await pool.query(
    'SELECT * FROM books WHERE id = ?', [id]
  );
  res.json(rows[0]);
});
```

books.js - Part 6: Server startup

```
app.listen(3001, () => {
  console.log('Server running on port 3001');
  console.log('Books API ready - connected to MySQL!');
});
```

API Endpoints Summary

Method	URL	Body	Purpose
GET	/books	None	List all books
GET	/books/:id	None	Get one book
POST	/books	Full JSON	Create new book
PUT	/books/:id	Full JSON	Replace book completely
PATCH	/books/:id	Partial JSON	Update selected fields

Complete Testing Guide

Full sequence to test everything end-to-end:

Step 1

Fetch All Books

GET

`http://localhost:3001/books`

Expected: Array of 15+ books with status 200

Step 2

Fetch One Book

GET

`http://localhost:3001/books/1`

Expected: Single book object (Wings of Fire) with status 200

Step 3

Test Not Found

GET

`http://localhost:3001/books/999`

Expected: {"message": "Book not found"} with status 404

Step 4

Create New Book

POST

`http://localhost:3001/books`

Request body

```
{
  "title": "Test Book",
  "author": "Test Author",
  "genre": "test",
  "pages": 100,
  "rating": 7
}
```

Expected: Book with auto-generated ID, status 201

Step 5

Full Update

PUT

`http://localhost:3001/books/16`

Request body (all fields required)

```
{
  "title": "Updated Title",
  "author": "Updated Author",
  "genre": "updated genre",
  "pages": 200,
  "rating": 9
}
```

Expected: Complete updated book with status 200

Step 6**Partial Update****PATCH****`http://localhost:3001/books/16`****Request body (any subset of fields)**

```
{
  "rating": 10
}
```

Expected: Complete book with only rating changed, status 200

Step 7**Verify in MySQL****Direct database check**

```
mysql> USE my_first_db;
mysql> SELECT * FROM books ORDER BY id DESC LIMIT 5;
```

Should show your recent changes reflected in MySQL.

Common Errors and Solutions

Error: ExecutionPolicy in PowerShell

Error

```
npm : File C:\...\npm.ps1 cannot be loaded because running scripts
is disabled on this system.
```

Solution:

Fix once, works forever

```
PS ...> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
Press Y and Enter
```

Error: Access denied for user root

Error

```
Error: Access denied for user 'root'@'localhost' (using password: YES)
```

Cause: Wrong MySQL password in connection pool config.

Solution:

- Open books.js
- Find password: 'YOUR_PASSWORD_HERE'
- Replace with your actual MySQL root password
- Save file (Ctrl + S)
- Restart server (Ctrl + C, then node books.js)

Error: Cannot find module mysql2/promise

Error

```
Error: Cannot find module 'mysql2/promise'
```

Cause: mysql2 package not installed.

Solution:

Install missing package

```
PS ...\books-api> npm install mysql2
```

Error: Unknown database my_first_db

Error

```
Error: Unknown database 'my_first_db'
```

Cause: Database does not exist in MySQL.

Solution:

Create database

```
mysql> CREATE DATABASE my_first_db;
mysql> USE my_first_db;
mysql> -- Now create books table and add data
```

Error: Cannot find module books.js

Error

```
Error: Cannot find module 'D:\...\books-api\books.js'
```

Possible Causes:

- Typo in filename (typed book.js instead of books.js)
- File is in wrong folder (outside books-api)
- File was not saved after creation

Solution:

- Run dir to see actual filename
- Check file exists in books-api folder
- Rename if needed (right-click in VS Code)
- Save file with Ctrl + S

Error: EADDRINUSE port 3001 already in use

Error

```
Error: listen EADDRINUSE: address already in use :::3001
```

Cause: Previous server instance still running.

Solution:

- Find the terminal where server is running
- Press Ctrl + C to stop it
- Then try again
- Or change port to 3002 in your code

The Request Flow Explained

POST Request Flow (Adding a New Book)

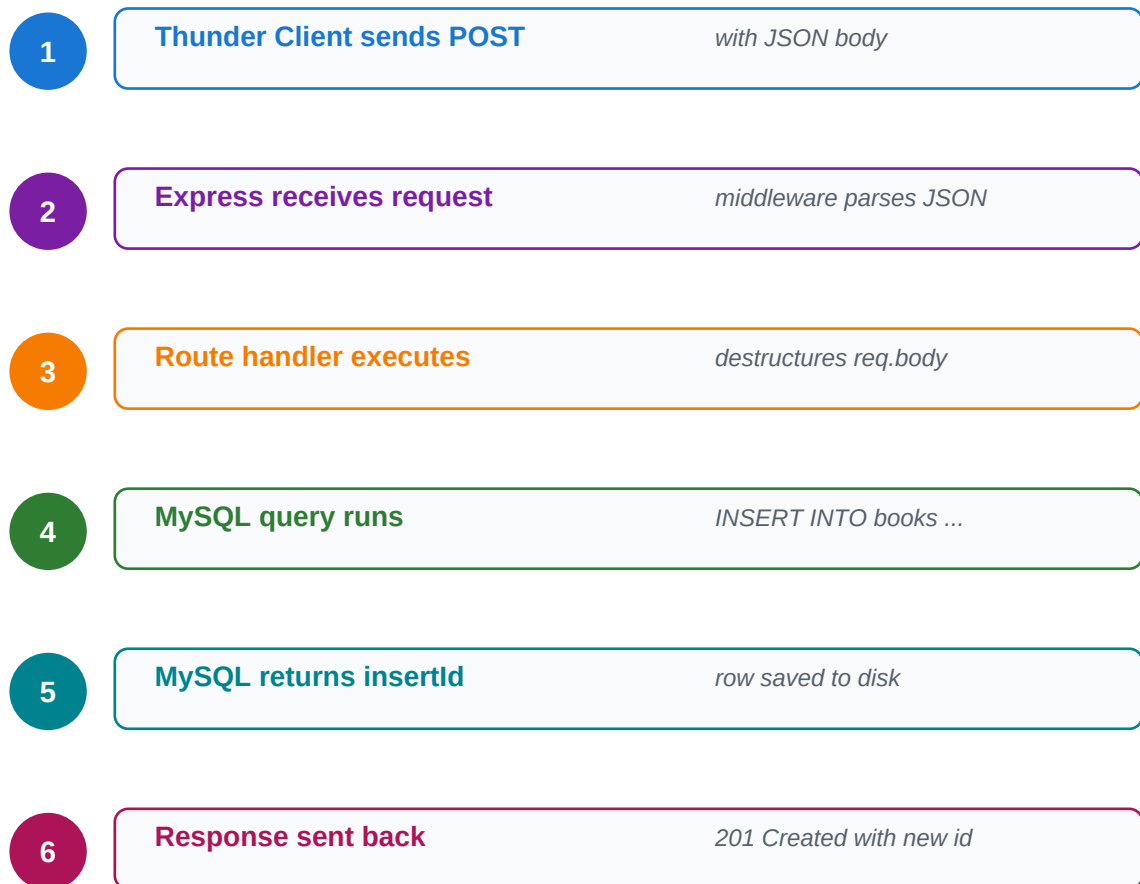


Figure 3: Detailed POST request flow with all six phases

Key Concepts Mastered

Through this tutorial, you have mastered these production-quality concepts:

Concept	Why It Matters
Connection pooling	Efficient resource use, high performance
async/await	Modern JavaScript async pattern
Prepared statements	Prevents SQL injection attacks
URL parameters	RESTful API design
HTTP status codes	Proper API communication
Error handling	Graceful failure responses
Destructuring	Clean code for extracting data
Dynamic query building	Flexible PATCH implementation

Best Practices Applied

- **Always use prepared statements** - never concatenate SQL
- **Always handle not-found cases** - return 404, do not crash
- **Always use appropriate status codes** - 200, 201, 400, 404
- **Always check affectedRows** - confirm updates worked
- **Always return updated data** - client can verify
- **Always use async/await** - not callbacks
- **Always separate concerns** - middleware, routes, config

What You Cannot Do Yet (Coming Soon)

This tutorial covers CRUD basics. Advanced topics for later:

- **Authentication** - JWT tokens, login/logout
- **Password hashing** - bcrypt for security
- **File uploads** - book cover images
- **Search and pagination** - handling large datasets
- **Environment variables** - hiding passwords in .env
- **Error middleware** - centralized error handling
- **Input validation** - Joi or Zod for data validation
- **Testing** - Jest, Supertest for automated tests
- **ORM libraries** - Sequelize or Prisma
- **Deployment** - hosting on cloud servers

What You Have Achieved

This tutorial documented a complete Node.js and MySQL integration project. Everything from scratch to a working REST API with 5 endpoints. Let us appreciate what has been built:

Technical Skills Gained

- Node.js project setup with npm
- Package installation and management
- MySQL driver configuration (mysql2)
- Connection pool creation
- Express route handlers with async/await
- Request parameter extraction
- Request body destructuring
- SQL prepared statements for security
- Complete CRUD implementation
- RESTful API design principles
- HTTP status code usage
- Error handling patterns
- Dynamic SQL query building
- Thunder Client for API testing

The Complete API Built

Endpoint	Purpose	HTTP Method
/books	List all	GET
/books/:id	Get one	GET
/books	Create new	POST
/books/:id	Full update	PUT
/books/:id	Partial update	PATCH

The Journey Recap

- **Started with:** Empty folder, no code
- **Built:** Complete REST API with 5 endpoints
- **Connected:** Node.js to MySQL for persistent data
- **Tested:** Every endpoint with Thunder Client
- **Verified:** Data persistence in MySQL directly
- **Learned:** Production-quality patterns and security

Where This Fits in Full Stack Development

You Now Have the Backend: You have built the complete BACKEND for a books application. Add a React frontend on top of this API, and you have a full stack application. Deploy it to a cloud server and it becomes a real production website. This tutorial covered the hardest and most valuable part.

Recommended Next Steps

1. **Add DELETE endpoint** - complete the CRUD (only D missing)
2. **Add input validation** - reject bad data
3. **Add error middleware** - catch unexpected errors
4. **Add environment variables** - .env file for password
5. **Add more endpoints** - search, filter, pagination
6. **Build a React frontend** - connect to this API
7. **Learn authentication** - JWT tokens for security
8. **Deploy to cloud** - Railway, Render, or Vercel

Congratulations! You have successfully built your first Node.js + MySQL integration. This is production-quality backend development. The patterns you learned here apply to any language and any database. You can now confidently work on real backend projects.

Node.js + MySQL Integration Tutorial

MCA Full Stack - Complete Live Coding Session Documentation